34

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

통합 테스트와 Testcontainers

제2부 NestJS 핵심 · 8장 테스트 기초

진짜 DB와 같이 돌려야 보이는 차이



© 한상훈 NestJS 백엔드 마스터

뜨지 않는 매진 표시

터미널

```
$ curl -X POST localhost:4034/performances/1/seats
{"performanceId":1,"count":60}
$ curl localhost:4034/performances/1/seats/summary
{"performanceId":1,"available":"60","soldOut":false}

(60석을 창구에서 전부 판매, 헤더 x-staff-key: key-kim)

$ curl localhost:4034/performances/1/seats/summary
{"performanceId":1,"available":"0","soldOut":false}
$ curl -X POST localhost:4034/performances/1/seats/A-3-15/hold
{"seatId":"A-3-15","seatStatus":"SOLD","type":"https://api.ticketing.example/problems/seat-unavailable",
 "title":"SeatUnavailableError","status":409,"detail":"A구역 3열 15번은 이미 팔렸거나 선택된 좌석입니다",
 "instance":"/performances/1/seats/A-3-15/hold"}
```

→ 서버: `DATABASE_URL`로 PostgreSQL 컨테이너(`ch08-pg-34-app`)에 연결

→ 좌석은 다 팔렸는데 `soldOut` false, `available`은 따옴표 붙은 `"0"`

→ 매진이 안 뜨면: 고객이 좌석 화면을 계속 새로고침, 끝난 공연 때문에 서버 부하 증가

© 한상훈 NestJS 백엔드 마스터

유닛 테스트는 전부 통과

```
  it('남은 좌석이 없으면 매진', async () => {
    for (let row = 1; row <= 3; row++) {
      for (let no = 1; no <= 20; no++) await service.sell(1, `A-${row}-${no}`);
    }

    expect(await service.summary(1)).toEqual({ performanceId: 1, available: 0, soldOut: true });
  });
```

○ ○ ○ 터미널

```
$ npx vitest run
 Test Files  10 passed (10)
      Tests  18 passed (18)
```

→ 유닛 테스트의 저장소: InMemorySeatsRepository (메모리 페이크)

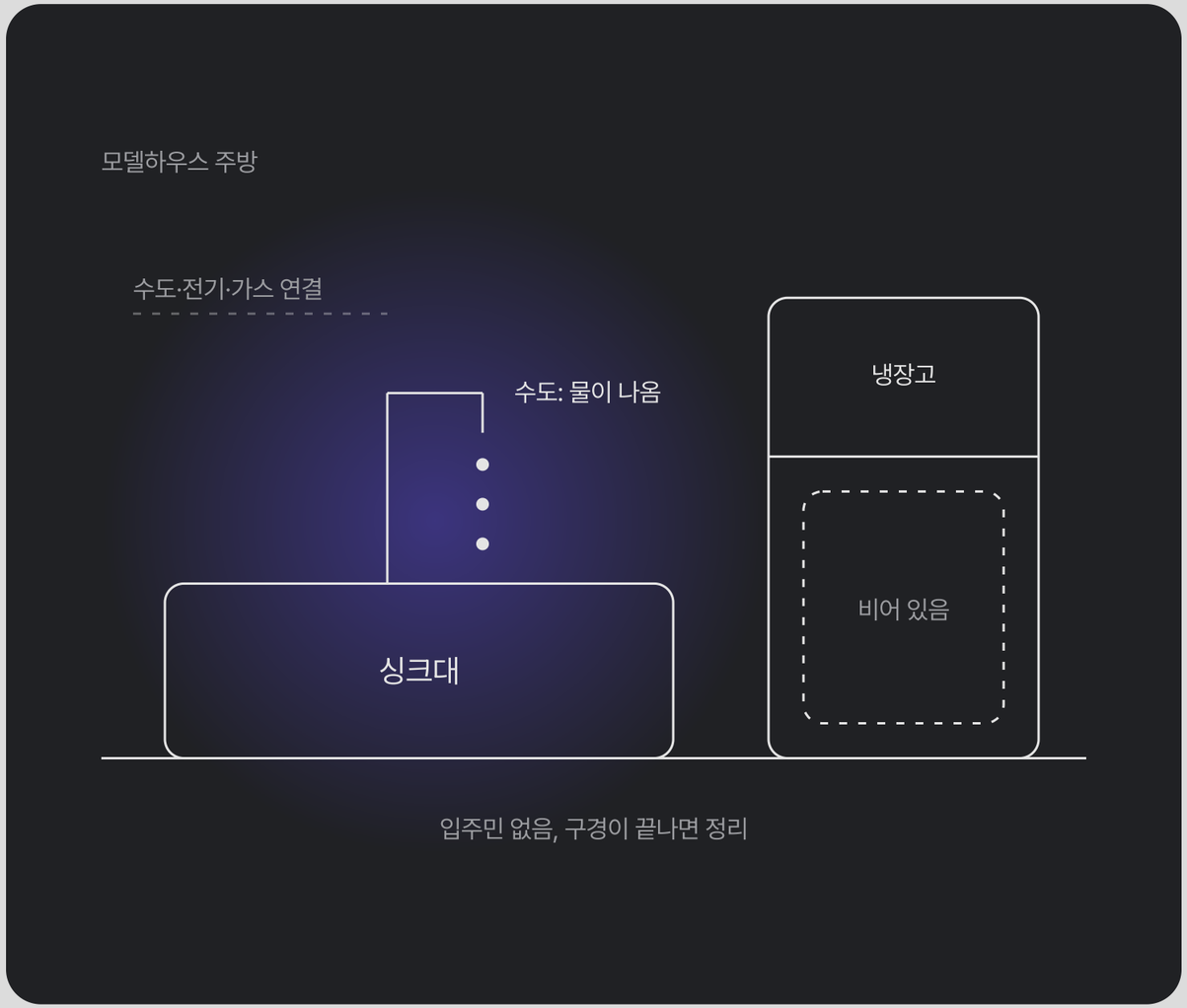
© 한상훈 NestJS 백엔드 마스터

모델하우스

→ 진짜 아파트와 같은 수도, 전기, 가스 연결

→ 입주민 없음, 구경이 끝나면 정리

→ 통합 테스트: 내 코드에 진짜 DB를 연결, 테스트용 데이터만



© 한상훈 NestJS 백엔드 마스터

저장소 두 개

```ts
export abstract class SeatsRepository {
  abstract insertMany(seats: Seat[]): Promise<void>;
  abstract findAll(performanceId: number): Promise<Seat[]>;
  abstract findOne(performanceId: number, seatId: string): Promise<Seat | undefined>;
  abstract changeIfAvailable(performanceId: number, seatId: string, status: SeatStatus): Promise<boolean>;
  abstract changeIfSold(performanceId: number, seatId: string, status: SeatStatus): Promise<boolean>;
  abstract countAvailable(performanceId: number): Promise<number>;
}
```

```ts
// seats.module.ts
    {
      provide: SeatsRepository,
      useFactory: () =>
        process.env.DATABASE_URL
          ? new PgSeatsRepository(new Pool({ connectionString: process.env.DATABASE_URL }))
          : new InMemorySeatsRepository(),
    },
```

→ `InMemorySeatsRepository`: Map에 저장, DB 없이 띄울 때와 유닛 테스트용

→ `PgSeatsRepository`: pg 드라이버로 PostgreSQL에 저장

→ `SeatsService`: 저장소를 생성자로 받음, 메서드가 전부 async로

→ `changeIfSold`: 팔린 좌석만 바꿈 (매표소 판매 취소)

© 한상훈 NestJS 백엔드 마스터

PgSeatsRepository

```
  async changeIfAvailable(performanceId: number, seatId: string, status: SeatStatus) {
    const { rowCount } = await this.pool.query(
      `UPDATE seats SET status = $3 WHERE performance_id = $1 AND id = $2 AND status = 'AVAILABLE'`,
      [performanceId, seatId, status],
    );
    return rowCount === 1;
  }

  async countAvailable(performanceId: number) {
    const { rows } = await this.pool.query<{ count: number }>(
      `SELECT count(*) FROM seats WHERE performance_id = $1 AND status = 'AVAILABLE'`,
      [performanceId],
    );
    return rows[0].count;
  }
```

→ query<{ count: number }>: 결과 모양을 적어 두는 타입 표시

© 한상훈 NestJS 백엔드 마스터

좌석 테이블

```sql
-- db/schema.sql
CREATE TABLE seats (
  performance_id integer NOT NULL,
  id text NOT NULL,
  zone text NOT NULL,
  row_no integer NOT NULL,
  seat_no integer NOT NULL,
  status text NOT NULL,
  PRIMARY KEY (performance_id, id)
);
```

→ 기본 키: 회차 번호 + 좌석 id

© 한상훈 NestJS 백엔드 마스터

통합 테스트

→ 통합 테스트(integration test): 내 코드와 진짜 외부 시스템을 같이 돌려, 둘 사이의 약속이 맞는지 보는 테스트

→ 이 강의 대상: 좌석 서비스 + PostgreSQL 저장소 + 진짜 PostgreSQL

→ 유닛 테스트가 못 보는 것: SQL 문법, 드라이버가 돌려주는 타입, 제약 조건, 트랜잭션

→ 예: 쿼리의 `seat_no`를 `seat_num`으로 잘못 적음 → 유닛 테스트 통과, 배포 후 첫 요청에서 에러

→ 유닛 테스트보다 느림: DB를 띄우고 네트워크를 탐

© 한상훈 NestJS 백엔드 마스터

어느 DB에 대고 돌리나

DB	문제	예
팀이 같이 쓰는 개발 DB	남의 데이터에 결과가 흔들림, TRUNCATE가 남의 데이터를 지움	동료가 넣은 좌석 때문에 개수가 다름
내 노트북에 깐 DB	운영과 버전·설정이 다름, 사람마다 다름	내 PC는 통과, 동료 PC는 실패
SQLite 같은 대용 DB	문법, 타입, 제약 동작이 다름	count의 타입, 기본 키 위반 메시지
docker compose로 띄워 둔 테스트 DB	먼저 띄워 둬야 함, 데이터가 남음, 동시 실행 때 공유	버전은 맞춤
테스트마다 띄우는 컨테이너	Docker 필요, 띄우는 시간	운영과 같은 버전을 매번 새로

© 한상훈 NestJS 백엔드 마스터

Testcontainers

→ Testcontainers: 테스트 코드에서 Docker 컨테이너를 띄우고, 끝나면 지우는 라이브러리

→ 운영과 같은 이미지, 같은 버전 (예: `postgres:17-alpine`)

→ 테스트마다 빈 DB, 매번 다른 포트

→ 필요한 것: Docker가 도는 PC나 CI

→ Ryuk: 테스트 프로세스가 죽어도 남은 컨테이너를 치우는 보조 컨테이너 (기본으로 켜짐)

→ `withName`: `docker ps`에서 알아보기 쉬움, 같은 이름이 있으면 시작 실패 → 동시 실행 때는 빼기

© 한상훈 NestJS 백엔드 마스터

통합 테스트 코드

```
describe('SeatsService + PostgreSQL', () => {
  let container: StartedPostgreSqlContainer;
  let pool: Pool;
  let service: SeatsService;

  beforeAll(async () => {
    container = await new PostgreSqlContainer('postgres:17-alpine').withName('ch08-pg-34').start();
    pool = new Pool({ connectionString: container.getConnectionUri() });
    await pool.query(readFileSync('db/schema.sql', 'utf8'));
  });

  afterAll(async () => {
    await pool.end();
    await container.stop();
  });

  beforeEach(async () => {
    await pool.query('TRUNCATE seats');
    service = new SeatsService(performances, new PgSeatsRepository(pool));
    await service.open(1);
  });
```

→ 회차 서비스 자리: 유닛 테스트와 같은 스텁

© 한상훈 NestJS 백엔드 마스터

통합 테스트 설정

```ts
// vitest.config.int.ts
export default defineConfig({
  plugins: [tsconfigPaths()],
  test: {
    globals: true,
    root: './',
    include: ['**/*.int-spec.ts'],
    hookTimeout: 60_000,
  },
});
```

```
"test:int": "vitest run --config ./vitest.config.int.ts"
```

→ 파일 이름: `*.int-spec.ts`, `npm test`(`*.spec.ts`)에는 안 섞임

→ `hookTimeout`: 컨테이너를 띄우는 `beforeAll`이 기본 10초를 넘을 수 있어서

→ 강사 PC 컨테이너 준비: 한가할 때 2초 남짓, 다른 작업과 겹칠 때 13~29초

© 한상훈 NestJS 백엔드 마스터

첫 실행

터미널

```
$ npm run test:int
컨테이너 준비: 2649ms, postgres://test:test@localhost:32768/test
 ✓ SeatsService + PostgreSQL > 빈 좌석을 붙잡으면 HELD가 된다 98ms
 ✓ SeatsService + PostgreSQL > 이미 붙잡힌 좌석을 또 붙잡으면 409 78ms
 × SeatsService + PostgreSQL > 남은 좌석이 없으면 매진 69ms

- Expected
+ Received
  {
-   "available": 0,
+   "available": "0",
    "performanceId": 1,
-   "soldOut": true,
+   "soldOut": false,
  }
```

데이터 블록: 컨테이너 준비 시간과 포트는 실행마다 다름(강사 PC 2.0~2.6초, 다른 작업과 겹칠 때 13.5~29.0초), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

count는 문자열로

→ PostgreSQL `count(*)`의 타입: `bigint` (64비트 정수)

→ JavaScript `number`: 2^53 넘는 정수는 정확히 표현 불가

→ node-postgres: `bigint`를 기본으로 문자열로 반환

→ `query<{ count: number }>`: 확인 없는 타입 표시, 실행 때 문자열 그대로

→ `"0" === 0` → false → `soldOut` false

→ 운영에서 먼저 만나면: 에러 로그 없음, 응답 200, JSON의 따옴표 하나로만 보임

© 한상훈 NestJS 백엔드 마스터

고친 쿼리

```
    const { rows } = await this.pool.query<{ count: number }>(
      `SELECT count(*)::int AS count FROM seats WHERE performance_id = $1 AND status = 'AVAILABLE'`,
      [performanceId],
    );
```

○ ○ ○ 터미널

```
✓ SeatsService + PostgreSQL > 빈 좌석을 붙잡으면 HELD가 된다 96ms
✓ SeatsService + PostgreSQL > 이미 붙잡힌 좌석을 또 붙잡으면 409 91ms
✓ SeatsService + PostgreSQL > 남은 좌석이 없으면 매진 88ms

    Tests  3 passed (3)
```

→ `::int`: DB에서 integer로 바꿔서 반환, 드라이버가 number로

→ 다른 방법: 코드에서 `Number()`, 드라이버의 타입 변환 설정

→ 정말 커질 수 있는 값 (금액 합계, 누적 방문 수): 문자열로 받아 큰 수 타입으로

© 한상훈 NestJS 백엔드 마스터

두 번 열기

저장소	같은 회차 좌석을 두 번 열면	결과
`InMemorySeatsRepository`	60석을 또 추가	120석, 남은 좌석 119 `{"performanceId":1,"available":119,"soldOut":false}`
`PgSeatsRepository`	기본 키 위반	로그에 `duplicate key value violates unique constraint "seats_pkey"`, 응답 500 (detail '요청을 처리하지 못했습니다.')

→ 같은 서비스 코드, 저장소만 다름

→ 메모리 쪽: 에러 없이 틀린 상태

© 한상훈 NestJS 백엔드 마스터

서비스에서 막기

```
  async open(performanceId: number) {
    this.performancesService.findOne(performanceId);
    if ((await this.seatsRepository.findAll(performanceId)).length > 0) {
      throw new SeatsAlreadyOpenError(`회차 ${performanceId}번 좌석은 이미 열려 있습니다`, { performanceId });
    }
```

```
it('같은 회차 좌석을 두 번 열면 409', async () => {
  await expect(service.open(1)).rejects.toThrow('회차 1번 좌석은 이미 열려 있습니다');
});
```

○ ○ ○ 터미널

```
$ npm run test:int
 ✓ SeatsService + PostgreSQL > 같은 회차 좌석을 두 번 열면 409 83ms
     Tests  4 passed (4)
```

→ `SeatsAlreadyOpenError`: 좌석 충돌과 같은 도메인 에러(kind conflict), 응답 409

→ 같은 테스트를 유닛(메모리)과 통합(PostgreSQL) 양쪽에

→ 기본 키: 서비스 검사를 빠져나간 경우의 마지막 방어선

© 한상훈 NestJS 백엔드 마스터

컨테이너와 데이터 정리

→ 컨테이너: 파일마다 한 번 (`beforeAll`), 띄우는 데 초 단위

→ 데이터: 테스트마다 TRUNCATE (`beforeEach`)

→ 끝나면: Pool 닫기 → 컨테이너 멈추기 (`afterAll`)

→ 테스트끼리 데이터가 남으면: 앞 테스트에 따라 뒤 테스트 결과가 바뀜

→ 다른 방법: 테스트마다 트랜잭션을 열고 롤백 (코드가 자기 트랜잭션을 쓰면 맞지 않음)

© 한상훈 NestJS 백엔드 마스터

유닛과 통합

	유닛 테스트	통합 테스트
저장소	메모리 페이크	PostgreSQL 컨테이너
걸리는 시간	밀리초	컨테이너 준비 초 단위 + 쿼리마다 수십 ms
필요한 것	Node	Node + Docker
잡는 것	업무 규칙 (409, 매진 판단)	SQL, 드라이버 타입, 제약 조건
명령	`npm test`	`npm run test:int`

→ 통합 테스트로 볼 코드: DB에 닿는 저장소, SQL을 쓰는 곳

→ 실행 시점: 유닛은 커밋마다, 통합은 PR·CI에서 (CI 러너에 Docker 필요)

→ 통합 테스트가 한꺼번에 실패: 먼저 Docker 실행 여부 확인

→ 서버 한 대의 메모리 좌석표: 서버 두 대면 좌석표도 두 개 → DB로 옮긴 이유

© 한상훈 NestJS 백엔드 마스터

통합 테스트의 규칙

"DB에 닿는 코드는 진짜 DB와 같이 돌려봐야, 가짜가 숨긴 차이가 보입니다."